SEGUNDO DEPARTAMENTAL- EJERCICIO 6- PILA

En este programa desarrollamos una pila dinámica, elobjetivo es permitir almacenar, insertar, eliminar, consultar y visualizar elementos enteros siguiendo el principio LIFO, que significa que el último elemento en entrar es el primero en salir. Para lograrlo, creamos dos clases principales: Nodo, que representa cada elemento de la pila y contiene un dato y un apuntador al siguiente nodo, y Pila, que se encarga de administrar la estructura mediante un apuntador llamado tope, a diferencia de una pila estática, nuestra implementación utiliza memoria dinámica mediante las instrucciones new y delete, por lo que no establecemos una capacidad máxima de elementos, también incorporamos un menú interactivo que nos permite realizar las operaciones PUSH, POP y PEEK, mostrar los elementos, verificar si la pila está vacía, limpiar la pantalla y finalizar el programa.

Errores y Correcciones: En la versión inicial, al momento de ejecutar en repetidas ocasiones alguna opción del menú la pantalla se llenaba información lo que causaba confusión al momento de comprobar la operación solicitada. Por lo que agregamos una nueva opción al menú mediante la estructura switch: “Limpiar pantalla”.

Código Completo:

```
#include <iostream>
#include <stdlib.h>
using namespace std;

// Clase que representa cada nodo de la pila
class Nodo {
public:
    int dato;
    Nodo* siguiente;

    Nodo(int valor) {
        dato = valor;
        siguiente = NULL;
    }
};

// Clase que maneja la pila dinamica
class Pila {
private:
    Nodo* tope;

public:
    // Constructor
    Pila() {
        tope = NULL;
    }

    // Destructor: libera la memoria de todos los nodos
    ~Pila() {
        while (tope != NULL) {
            Nodo* auxiliar = tope;
            tope = tope->siguiente;
```

```
            delete auxiliar;
        }
    }

    // Insertar elemento (PUSH)
    void insertar(int valor) {
        Nodo* nuevo = new Nodo(valor);

        nuevo->siguiente = tope;
        tope = nuevo;

        cout << "Elemento insertado correctamente.\n";
    }

    // Eliminar elemento (POP)
    void eliminar() {
        if (tope == NULL) {
            cout << "La pila esta vacia.\n";
        } else {
            Nodo* auxiliar = tope;

            cout << "Elemento eliminado: "
                 << tope->dato << endl;

            tope = tope->siguiente;
            delete auxiliar;
        }
    }

    // Mostrar elementos de la pila
    void mostrar() {
        if (tope == NULL) {
            cout << "La pila esta vacia.\n";
        } else {
            Nodo* actual = tope;

            cout << "\nElementos de la pila:\n";

            while (actual != NULL) {
                cout << actual->dato << endl;
                actual = actual->siguiente;
            }
        }
    }

    // Consultar el elemento superior (PEEK)
    void consultarTope() {
        if (tope == NULL) {
            cout << "La pila esta vacia.\n";
        } else {
            cout << "Elemento en el tope: "
                 << tope->dato << endl;
        }
    }

    // Verificar si la pila esta vacia
    bool estaVacia() {
        return tope == NULL;
    }
};

int main() {
    Pila pila;
```

```
    int opcion, valor;

    do {
        cout << "\n=== PILA DINAMICA EN POO ===\n";
        cout << "1. Insertar elemento (PUSH)\n";
        cout << "2. Eliminar elemento (POP)\n";
        cout << "3. Mostrar pila\n";
        cout << "4. Consultar tope (PEEK)\n";
        cout << "5. Verificar si esta vacia\n";
        cout << "6. Limpiar pantalla\n";
        cout << "7. Salir\n";
        cout << "Selecciona una opcion: ";
        cin >> opcion;

        switch (opcion) {
            case 1:
                cout << "Ingresa un numero: ";
                cin >> valor;
                pila.insertar(valor);
                break;

            case 2:
                pila.eliminar();
                break;

            case 3:
                pila.mostrar();
                break;

            case 4:
                pila.consultarTope();
                break;

            case 5:
                if (pila.estaVacia())
                    cout << "La pila esta vacia.\n";
                else
                    cout << "La pila contiene elementos.\n";
                break;

            case 6:
                system("cls");
                break;

            case 7:
                cout << "Programa finalizado.\n";
                break;

            default:
                cout << "Opcion no valida.\n";
        }

    } while (opcion != 7);

    return 0;
}
```

Explicación del Código:

1. Declaración de la Clase Nodo (class Nodo)

Creamos la clase Nodo, cuya función es representar cada elemento que almacenamos dentro de nuestra pila dinámica.

2. Declaración de los Atributos Públicos (dato y siguiente)

- dato: es una variable de tipo entero que almacena el valor correspondiente al nodo.
- siguiente: es un apuntador de tipo Nodo* que guarda la dirección de memoria del siguiente nodo.

Utilizamos el apuntador siguiente para conectar los diferentes nodos y mantener organizada nuestra pila dinámica.

3. Declaración del Constructor de Nodo (Nodo(int valor))

Su función es inicializar los atributos de cada nuevo nodo, utilizamos NULL porque todavía no hemos establecido una conexión con otro nodo.

4. Declaración de la Clase Pila (class Pila)

Creamos la clase Pila, que se encarga de administrar todos los nodos que forman nuestra estructura de datos. Dentro de esta clase implementamos los métodos necesarios para insertar, eliminar, mostrar y consultar elementos, además de verificar si la pila está vacía, también incorporamos un constructor y un destructor para controlar la inicialización y liberación de la memoria utilizada.

5. Declaración del Atributo Privado (Nodo* tope)

Este atributo es un apuntador de tipo Nodo* que almacena la dirección de memoria del nodo que se encuentra en la parte superior de nuestra pila.

6. Declaración del Constructor de Pila (Pila())

Su función es inicializar el apuntador tope con el valor NULL, asignamos este valor porque al momento de crear nuestra pila todavía no existen nodos almacenados.

7. Método para Insertar Elementos (void insertar(int valor))

Creamos el método insertar(), cuya función es agregar un nuevo elemento a nuestra pila dinámica, esta operación también se conoce como PUSH, el método recibe un número entero mediante el parámetro valor y después utilizamos la instrucción new para reservar memoria dinámica y crear un nuevo objeto de tipo Nodo. La dirección de memoria de ese objeto se almacena en el apuntador nuevo.

8. Conexión del Nuevo Nodo con la Pila (nuevo->siguiente = tope)

Después de crear el nuevo nodo, establecemos su conexión con el elemento que actualmente se encuentra en la parte superior de nuestra pila, así el nuevo elemento se convierte en la parte superior de nuestra pila.

9. Método para Eliminar Elementos (void eliminar())

Su función es retirar el nodo que se encuentra en la parte superior de nuestra pila, esta operación también se conoce como POP, antes de realizar la eliminación, verificamos si nuestra estructura contiene elementos, utilizamos la condición tope == NULL para comprobar si la pila está vacía, si se cumple, mostramos un mensaje indicando que no existen elementos para eliminar y evitamos realizar operaciones sobre una dirección de memoria inexistente.

10. Eliminación del Nodo Superior (auxiliar, tope y delete)

De esta manera, retiramos el elemento superior y evitamos dejar memoria reservada que ya no necesitamos.

11. Método para Mostrar los Elementos (void mostrar())

Creamos el método mostrar(), cuya función es visualizar todos los elementos que se encuentran almacenados en nuestra pila. Primero verificamos si nuestra pila está vacía mediante la condición tope == NULL, si no existen elementos, mostramos un mensaje indicando que la pila está vacía, en caso contrario, recorremos los nodos para presentar sus valores.

12. Recorrido de los Nodos mediante el Apuntador (actual)

Cuando nuestra pila contiene elementos, declaramos un apuntador temporal llamado actual, inicializamos actual con la dirección del nodo superior y después utilizamos un ciclo while para recorrer todos los nodos, el ciclo termina cuando llegamos al final de la estructura, es decir, cuando actual toma el valor NULL.

13. Método para Consultar el Elemento Superior (void consultarTope())

Creamos el método consultarTope(), cuya función es mostrar el valor del elemento que se encuentra en la parte superior de nuestra pila sin eliminarlo, primero verificamos si nuestra pila está vacía, si tope tiene el valor NULL, mostramos un mensaje indicando que no existe ningún elemento para consultar.

14. Consulta del Elemento Superior (tope->dato)

Cuando nuestra pila contiene elementos, utilizamos tope->dato para acceder directamente al valor almacenado en el nodo superior, a diferencia de la operación POP, con PEEK únicamente mostramos el elemento y no modificamos la estructura, por lo tanto, después de realizar la consulta, nuestra pila conserva todos sus nodos y conexiones.

15. Método para Verificar si la Pila Está Vacía (bool estaVacia())

Creamos el método estaVacia(), cuya función es comprobar si nuestra pila contiene elementos, este método utiliza el tipo de dato bool, que permite devolver únicamente

dos posibles resultados y u tilizamos la instrucción return para devolver directamente el resultado de la comparación.

16. Función Principal (int main())

Dentro de esta función creamos un objeto llamado pila, perteneciente a nuestra clase Pila, al crear este objeto, se ejecuta automáticamente el constructor y se inicializa tope con el valor NULL.